A SMALL THEME FOR MODERN SLIDES

OWL, POS & External APIs

Reactive frontend, POS customization, XML-RPC

Weblearns Academy

Odoo Full-Stack Developer Program

Senior Developer Lecture Materials

July 2026

ROADMAP

Agenda

1. OWL components, state, and templates
2. Patching, rpc, and custom widgets
3. POS buttons, popups, and translations
4. XML-RPC authentication and CRUD

TOC

METHOD

How this deck is taught

PATTERN

Objectives → teaching → examples → pitfalls → lab

LIVE CODING

Type commands with learners; freeze a reference commit

SAFETY

Use disposable DBs; never demo sudo shortcuts in production

OWL POS RPC XML-RPC

1 W01 – W05

OWL, POS & External APIs · teaching block 1

W01 · OBJECTIVES

OWL Architecture and Reactive Rendering

- Explain how OWL components render from reactive state.
- Identify the role of services, registries, and templates.
- Avoid direct DOM manipulation in normal component logic.

> Lab target — Create a small client action that increments a counter and logs when the component rerenders.

W01 · TEACHING 1

OWL Architecture and Reactive Rendering

OWL Architecture and Reactive Rendering belongs to the client-side layer where user intent, browser state, and Odoo services meet. OWL is Odoo's component framework and provides the reactive mental model behind many modern web client features. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

OWL Architecture and Reactive Rendering (continued)

The design starts with ownership of state and boundaries. Decide which component owns state, which services provide data, and which children only render props. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

OWL Architecture and Reactive Rendering (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Use Component classes, static templates, setup methods, hooks, and the registry rather than creating global scripts. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

OWL Architecture and Reactive Rendering (continued)

Verification requires more than refreshing the page once. Inspect rerenders, service calls, and asset loading in browser developer tools after every meaningful change. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

OWL Architecture and Reactive Rendering (continued)

In production, owl architecture and reactive rendering decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A maintainable OWL customization feels native because it composes with the framework instead of fighting it. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

What to remember

- Reactive state changes trigger rerendering.
- Templates describe UI; components own behavior.
- Services centralize shared capabilities such as rpc and notifications.
- Registries let addons extend the web client.
- Direct DOM writes are a last resort.

W01 · EXAMPLE 1

Small reactive counter component

Small reactive counter component

The state object is reactive, so incrementing value rerenders the template that displays it.

```
/** @odoo-module **/

import { Component, useState } from "@odoo/owl";
import { registry } from "@web/core/registry";

export class TrainingCounter extends Component {
    static template = "training.TrainingCounter";

    setup() {
        this.state = useState({ value: 0 });
    }

    increment() {
        this.state.value += 1;
# ... truncated for slide
```

W01 · PITFALLS

Common mistakes

- Changing random DOM nodes instead of changing component state.
- Putting global variables in asset files for shared state.
- Forgetting to register the component under the correct registry category.
- Treating OWL like jQuery with templates attached.

> Caution — Validate fixes in a disposable database before production.

OWL building blocks

- Component
- Template
- State
- Services
- Registry

W02 · OBJECTIVES

Component Structure and Lifecycle Hooks

- Organize component files and templates cleanly.
- Use lifecycle hooks for setup and cleanup.
- Keep asynchronous loading predictable.

> Lab target — Build a dashboard component that loads statistics before render and refreshes every minute without leaking timers.

Component Structure and Lifecycle Hooks

Component Structure and Lifecycle Hooks belongs to the client-side layer where user intent, browser state, and Odoo services meet. Component structure matters because Odoo assets are bundled, upgraded, translated, and extended by other modules. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

Component Structure and Lifecycle Hooks (continued)

The design starts with ownership of state and boundaries. Keep JavaScript behavior, XML templates, and SCSS names aligned so another developer can find the complete feature quickly. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Component Structure and Lifecycle Hooks (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Use setup for hooks, onWillStart for initial asynchronous work, onMounted for browser-only integration, and onWillUnmount for cleanup. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Component Structure and Lifecycle Hooks (continued)

Verification requires more than refreshing the page once. Test navigation away from the component while requests are pending and verify that timers or listeners do not leak. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

Component Structure and Lifecycle Hooks (continued)

In production, component structure and lifecycle hooks decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Lifecycle discipline prevents slow memory leaks and race conditions that otherwise appear only during long back-office sessions. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W02 · KEY POINTS

What to remember

- setup is the main hook registration point.
- onWillStart can await data before first render.
- onMounted is safe for DOM-dependent integrations.
- onWillUnmount cleans up listeners and timers.
- Asset paths should mirror feature names.

W02 · EXAMPLE 1

Lifecycle with cleanup

Lifecycle with cleanup

The component loads initial data before rendering and removes its refresh timer when it is destroyed.

```
/** @odoo-module **/

import { Component, onMounted, onWillStart, onWillUnmount, useState } from "@odoo/owl";
import { useService } from "@web/core/utils/hooks";

export class CourseDashboard extends Component {
    static template = "training.CourseDashboard";

    setup() {
        this.orm = useService("orm");
        this.state = useState({ stats: null });
        let timer;

        onWillStart(async () => {
# ... truncated for slide
```

Common mistakes

- Starting intervals without clearing them.
- Calling browser APIs before the component is mounted.
- Loading data in the constructor instead of setup hooks.
- Ignoring navigation races during slow RPC responses.

> Caution — Validate fixes in a disposable database before production.

useState, useRef, Events, and Props

- Use state for mutable component data.
- Use refs for specific DOM elements.
- Pass behavior through events and props cleanly.

> Lab target — Create a child filter component that receives an onApply callback and prove it remains reusable in two parent components.

useState, useRef, Events, and Props

useState, useRef, Events, and Props belongs to the client-side layer where user intent, browser state, and Odoo services meet. OWL gives developers clear tools for local state, DOM references, parent-to-child data, and child-to-parent communication. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

useState, useRef, Events, and Props (continued)

The design starts with ownership of state and boundaries. Before writing code, decide whether a value is owned by the component, passed by the parent, derived from services, or simply stored in the DOM. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

useState, useRef, Events, and Props (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Use useState for reactive values, useRef for controlled DOM access, props for inputs, and callback props or triggered events for outputs. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

useState, useRef, Events, and Props (continued)

Verification requires more than refreshing the page once. Verify that a child component still works when props change and that refs are not accessed before mount. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

useState, useRef, Events, and Props (continued)

In production, usestate, useref, events, and props decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Correct ownership keeps components reusable and avoids the common trap of several components mutating the same data without a single source of truth. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W03 · KEY POINTS

What to remember

- State is mutable and reactive inside the component.
- Props are inputs from the parent.
- Refs point to DOM nodes after render.
- Events should describe user intent.
- Derived values should not be duplicated unnecessarily.

W03 · EXAMPLE 1

Filter input with ref and callback prop

Filter input with ref and callback prop

The component owns the input text, calls the parent callback when applied, and uses a ref only to restore focus.

```
/** @odoo-module **/

import { Component, useRef, useState } from "@odoo/owl";

export class CourseFilter extends Component {
    static template = "training.CourseFilter";
    static props = { onApply: Function };

    setup() {
        this.state = useState({ text: "" });
        this.inputRef = useRef("filterInput");
    }

    apply() {
# ... truncated for slide
```

Common mistakes

- Mutating props directly.
- Using refs as general state storage.
- Passing large objects when a small prop would do.
- Creating event names that describe implementation instead of intent.

> Caution — Validate fixes in a disposable database before production.

QWeb XML Directives: t-if, t-foreach, and t-model

- Render conditional content in OWL XML templates.
- Loop over arrays safely with keys.
- Bind form controls with t-model where appropriate.

> Lab target — Write a template that lists courses, shows an empty state, and filters from a t-model input stored in component state.

QWeb XML Directives: t-if, t-foreach, and t-model

QWeb XML Directives: t-if, t-foreach, and t-model belongs to the client-side layer where user intent, browser state, and Odoo services meet. OWL templates use QWeb-style XML directives to describe dynamic browser UI. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

QWeb XML Directives: t-if, t-foreach, and t-model (continued)

The design starts with ownership of state and boundaries. Keep templates declarative: conditions should select presentation, loops should render prepared lists, and form bindings should update local state. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

QWeb XML Directives: t-if, t-foreach, and t-model (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Use t-if for conditions, t-foreach with t-as and t-key for repeatable data, and t-model for simple two-way form input binding. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

QWeb XML Directives: t-if, t-foreach, and t-model (continued)

Verification requires more than refreshing the page once. Test empty arrays, duplicate labels, and rapid typing because template mistakes often appear as unstable DOM updates. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

QWeb XML Directives: t-if, t-foreach, and t-model (continued)

In production, qweb xml directives: t-if, t-foreach, and t-model decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Good templates are readable enough that a reviewer can understand the UI states without opening the JavaScript file first. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W04 · KEY POINTS

What to remember

- Every t-foreach should have a stable t-key.
- t-if removes or inserts template branches.
- t-model is useful for simple local form values.
- Complex formatting belongs in component methods or getters.
- XML must remain valid and escaped.

W04 · EXAMPLE 1

OWL template directives

OWL template directives

The template handles search input, empty state, and stable list rendering with a key per course.

```
<templates xml:space="preserve">
    <t t-name="training.CourseList">
        <input t-ref="filterInput" t-model="state.query" placeholder="Search courses"/>
        <p t-if="!state.courses.length">No courses found.</p>
        <ul>
            <li t-foreach="state.courses" t-as="course" t-key="course.id">
                <span t-esc="course.name"/>
                <span class="text-muted" t-esc="course.teacher"/>
            </li>
        </ul>
    </t>
</templates>
```

Common mistakes

- Looping without t-key.
- Putting business queries inside template expressions.
- Forgetting XML escaping for comparison operators.
- Using t-model against props instead of local state.

> Caution — Validate fixes in a disposable database before production.

W05 · OBJECTIVES

Custom Form Field Widgets

- Register a field widget in the web client.
- Respect standard field props and update behavior.
- Use widgets to improve editing without changing model semantics.

> Lab target — Create a five-star rating widget for an integer field and test it in readonly and editable form modes.

Custom Form Field Widgets

Custom Form Field Widgets belongs to the client-side layer where user intent, browser state, and Odoo services meet. A custom field widget changes how one field is displayed or edited while keeping the model field as the source of truth. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

Custom Form Field Widgets (continued)

The design starts with ownership of state and boundaries. Use a widget when the default field component is correct semantically but not expressive enough for the workflow. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Custom Form Field Widgets (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Register the widget in the fields registry, accept standard props, and call props.update when the user changes the value. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Custom Form Field Widgets (continued)

Verification requires more than refreshing the page once. Test readonly mode, required fields, invalid values, and list/form contexts because widgets may be reused outside the original screen. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

Custom Form Field Widgets (continued)

In production, custom form field widgets decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production widgets should be boringly compatible with Odoo's form renderer: they should not bypass validation, dirty state, or onchange behavior. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W05 · KEY POINTS

What to remember

- Field widgets are registered in the fields registry.
- props.value is the current field value.
- props.update writes a new value to the record buffer.
- Support readonly mode explicitly.
- Widgets should not own persistent business state.

W05 · EXAMPLE 1

Rating field widget

Rating field widget

The widget follows the standard field contract and updates the form record through props.update.

```
/** @odoo-module **/

import { Component } from "@odoo/owl";
import { registry } from "@web/core/registry";
import { standardFieldProps } from "@web/views/fields/standard_field_props";

export class RatingField extends Component {
    static template = "training.RatingField";
    static props = { ...standardFieldProps };

    setRating(value) {
        if (!this.props.readonly) {
            this.props.update(value);
        }
# ... truncated for slide
```

Common mistakes

- Writing directly to the server from the widget instead of using props.update.
- Ignoring readonly mode.
- Registering a widget with a generic key likely to collide.
- Supporting field types the component cannot handle.

> Caution — Validate fixes in a disposable database before production.

2 W06 – W10

OWL, POS & External APIs · teaching block 2

Overriding JavaScript Components with the Patch Utility

- Patch existing components narrowly.
- Call super behavior when extending methods.
- Minimize upgrade risk when overriding Odoo web code.

> Lab target — Patch a small controller method to emit a custom event, then remove the patch and confirm the rest of the feature is isolated.

Overriding JavaScript Components with the Patch Utility

Overriding JavaScript Components with the Patch Utility belongs to the client-side layer where user intent, browser state, and Odoo services meet. The patch utility is the preferred way to adjust existing JavaScript classes when registries or props are not enough. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

Overriding JavaScript Components with the Patch Utility (continued)

The design starts with ownership of state and boundaries. Patch only the method or getter you need, and first ask whether a registry extension, service, or template inheritance would be safer. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Overriding JavaScript Components with the Patch Utility (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Import the target class and patch it with a small object that adds or extends behavior while preserving upstream logic where required. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Overriding JavaScript Components with the Patch Utility (continued)

Verification requires more than refreshing the page once. Test against the exact Odoo version and read upstream changes during upgrades because patches depend on private implementation details more than normal extensions. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

Overriding JavaScript Components with the Patch Utility (continued)

In production, overriding javascript components with the patch utility decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A good patch is almost invisible: one responsibility, no global side effects, and an easy removal path when upstream gains the needed hook. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

What to remember

- Patch is powerful and should be narrow.
- Prefer public extension points first.
- Call super when preserving behavior matters.
- Version upgrades can break patches.
- Name modules and methods clearly for traceability.

W06 · EXAMPLE 1

Patch a method to add a training flag

Patch a method to add a training flag

The patch adds a small event before save while delegating the actual save behavior to the original controller.

```
/** @odoo-module **/

import { patch } from "@web/core/utils/patch";
import { FormController } from "@web/views/form/form_controller";

patch(FormController.prototype, {
    async saveButtonClicked(params = {}) {
        this.env.bus.trigger("training:before-form-save", {
            resModel: this.props.resModel,
        });
        return super.saveButtonClicked(params);
    },
});
```

Common mistakes

- Patching large methods by copying upstream source.
- Forgetting to call super when extending behavior.
- Using patch when a registry extension exists.
- Not reviewing patches during every major upgrade.

> Caution — Validate fixes in a disposable database before production.

RPC and ORM Services

- Use orm and rpc services appropriately.
- Call model methods from OWL components.
- Handle loading and errors cleanly.

> Lab target — Replace a direct fetch call with orm.call and add a loading indicator plus a notification on failure.

RPC and ORM Services

RPC and ORM Services belongs to the client-side layer where user intent, browser state, and Odoo services meet. Odoo provides services so client code does not need to know every HTTP detail or session convention. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

RPC and ORM Services (continued)

The design starts with ownership of state and boundaries. Use orm for model operations and rpc for custom endpoints or lower-level calls where the ORM service does not fit. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

RPC and ORM Services (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Inject services with useService, keep request code in explicit methods, and store loading or error state so the UI communicates what is happening. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

RPC and ORM Services (continued)

Verification requires more than refreshing the page once. Test server access rights, record rules, network failures, and repeated clicks because the browser can make invalid calls faster than a human can diagnose them. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

RPC and ORM Services (continued)

In production, rpc and orm services decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Service-based code is easier to migrate because authentication, context, and error handling stay aligned with the framework. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

What to remember

- useService injects framework services.
- orm.call invokes model methods.
- rpc is useful for custom routes.
- Always represent loading and failure states.
- Server methods must still validate access.

W07 · EXAMPLE 1

Load dashboard data through orm

Load dashboard data through orm

The component uses framework services and exposes loading and error feedback to users.

```
/** @odoo-module **/

import { Component, useState } from "@odoo/owl";
import { useService } from "@web/core/utils/hooks";

export class TrainingDashboard extends Component {
    static template = "training.TrainingDashboard";

    setup() {
        this.orm = useService("orm");
        this.notification = useService("notification");
        this.state = useState({ loading: false, stats: {} });
    }

# ... truncated for slide
```

Common mistakes

- Calling fetch directly for ORM work.
- Ignoring errors and leaving stale UI state.
- Trusting client parameters without server validation.
- Making one RPC call per row when a batched method would do.

> Caution — Validate fixes in a disposable database before production.

W08 · OBJECTIVES

Custom Kanban and Dashboard OWL Views

- Design a dashboard as a client action or custom view.
- Register components in the correct registry.
- Keep dashboard data server-side and aggregated.

> Lab target — Build a client action dashboard with three KPI cards and one card that opens a filtered course list.

Custom Kanban and Dashboard OWL Views

Custom Kanban and Dashboard OWL Views belongs to the client-side layer where user intent, browser state, and Odoo services meet. Dashboards should present decisions, exceptions, and navigation paths rather than becoming a second database browser. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

Custom Kanban and Dashboard OWL Views (continued)

The design starts with ownership of state and boundaries. Choose a client action for a standalone dashboard and a custom view only when it truly replaces how users inspect a model. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Custom Kanban and Dashboard OWL Views (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Register the component, define an action tag, load aggregated data from model methods, and keep cards reusable where possible. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Custom Kanban and Dashboard OWL Views (continued)

Verification requires more than refreshing the page once. Verify performance on realistic data because dashboards are often opened by managers every morning and must not run expensive per-card queries. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

Custom Kanban and Dashboard OWL Views (continued)

In production, custom kanban and dashboard owl views decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A production dashboard is an operations surface: it needs clear ownership, stable metrics, and failure behavior that does not block the rest of Odoo. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W08 · KEY POINTS

What to remember

– Client actions are good for standalone dashboards.
– Custom views are deeper integrations with the view system.
– Aggregate on the server, render on the client.
– Use action tags and registries consistently.
– Dashboard cards should link to filtered records.

W08 · EXAMPLE 1

Client action registration

Client action registration

The action tag connects the server-side ir.actions.client record to the OWL component.

```
/** @odoo-module **/

import { registry } from "@web/core/registry";
import { TrainingDashboard } from "./training_dashboard";

registry.category("actions").add("training.dashboard", TrainingDashboard);

// XML action
// <record id="action_training_dashboard" model="ir.actions.client">
//     <field name="name">Training Dashboard</field>
//     <field name="tag">training.dashboard</field>
// </record>
```

Common mistakes

- Loading raw records in the browser and aggregating there.
- Creating dashboards without drill-down actions.
- Using a custom view when a client action is enough.
- Forgetting access rights on the server statistics method.

> Caution — Validate fixes in a disposable database before production.

W09 · OBJECTIVES

POS Custom Buttons and Click Events

- Add a custom control button to a POS screen.
- Handle click events with OWL methods.
- Keep POS actions compatible with offline behavior.

> Lab target — Add a POS ProductScreen button that shows a notification and disable it when no order is active.

POS Custom Buttons and Click Events

POS Custom Buttons and Click Events belongs to the client-side layer where user intent, browser state, and Odoo services meet. POS extensions run in a high-speed operational UI where every button must be obvious and resilient. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

POS Custom Buttons and Click Events (continued)

The design starts with ownership of state and boundaries. Decide which screen owns the action and whether the feature can work offline or must warn the cashier that connectivity is required. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

POS Custom Buttons and Click Events (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Register a button component, attach it to the target POS screen, and handle click events with methods that use POS services or models. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

POS Custom Buttons and Click Events (continued)

Verification requires more than refreshing the page once. Test with barcode scanning, touch screens, and rapid repeated clicks because POS users do not interact like back-office users. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

POS Custom Buttons and Click Events (continued)

In production, pos custom buttons and click events decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A good POS button adds one clear workflow step without slowing checkout or compromising order consistency. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W09 · KEY POINTS

What to remember

– POS buttons should be screen-specific.
– Click handlers should be idempotent where possible.
– Offline behavior must be considered.
– Use POS registries and services instead of globals.
– User feedback must be immediate.

W09 · EXAMPLE 1

Simple POS control button

Simple POS control button

The button is added to the ProductScreen and handles the click through a component method.

```
/** @odoo-module **/

import { Component } from "@odoo/owl";
import { ProductScreen } from "@point_of_sale/app/screens/product_screen/product_screen";

export class CourseInfoButton extends Component {
    static template = "training.CourseInfoButton";

    onClick() {
        this.env.services.notification.add("Training mode is active.", {
            type: "info",
        });
    }
}
# ... truncated for slide
```

Common mistakes

– Adding a button globally when it belongs to one screen.
– Not debouncing expensive click operations.
– Assuming a cashier has back-office permissions.
– Ignoring offline mode when the click needs the server.

> Caution — Validate fixes in a disposable database before production.

W10 · OBJECTIVES

Hiding POS Buttons Safely

- Hide or show POS UI based on configuration.
- Distinguish visibility from authorization.
- Keep hidden actions protected server-side.

> Lab target — Add a Boolean POS configuration field that controls a custom button and prove the server method still rejects unauthorized calls.

Hiding POS Buttons Safely

Hiding POS Buttons Safely belongs to the client-side layer where user intent, browser state, and Odoo services meet. Hiding a POS button can simplify cashier flow, but it must not be confused with security. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

Hiding POS Buttons Safely (continued)

The design starts with ownership of state and boundaries. Decide whether the control is hidden because it is irrelevant, disabled because a condition is missing, or forbidden because the user lacks a role. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Hiding POS Buttons Safely (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Use button conditions, configuration values loaded into POS, and server-side validation for any sensitive operation. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Hiding POS Buttons Safely (continued)

Verification requires more than refreshing the page once. Test every cashier role and POS configuration because a button hidden on one terminal may remain visible on another configuration. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

Hiding POS Buttons Safely (continued)

In production, hiding pos buttons safely decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production POS systems are distributed user interfaces; UI conditions must match central business policy and survive session reloads. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W10 · KEY POINTS

What to remember

– Visibility improves workflow but is not security.
– Button conditions can read POS configuration.
– Sensitive methods need server checks.
– Different POS configs may show different controls.
– Use clear disabled feedback when possible.

W10 · EXAMPLE 1

Configuration-based button condition

Configuration-based button condition

The button appears only for POS configurations where the feature is enabled.

```
ProductScreen.addControlButton({
    component: CourseInfoButton,
    condition() {
        const config = this.pos.config;
        return Boolean(config.module_training && config.show_training_button);
    },
});
```

Common mistakes

- Treating a hidden button as access control.
- Hard-coding user ids in JavaScript.
- Forgetting to load the configuration field into POS data.
- Making the button disappear without explaining why.

> Caution — Validate fixes in a disposable database before production.

3 W11 – W15

OWL, POS & External APIs · teaching block 3

W11 · OBJECTIVES

POS Popups and Dialog Flow

- Show confirmation and input popups in POS.
- Handle popup results correctly.
- Design cashier-friendly dialog flows.

> Lab target — Create a confirmation popup that asks before applying a manager discount or clearing an order.

W11 · TEACHING 1

POS Popups and Dialog Flow

POS Popups and Dialog Flow belongs to the client-side layer where user intent, browser state, and Odoo services meet. POS popups interrupt the fastest workflow in Odoo, so they must be short, intentional, and recoverable. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

POS Popups and Dialog Flow (continued)

The design starts with ownership of state and boundaries. Use a popup when the cashier needs confirmation, small input, or a warning that cannot be ignored before continuing. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

POS Popups and Dialog Flow (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Call the dialog or popup service, await the result where the framework supports it, and update the order only after the user confirms. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

POS Popups and Dialog Flow (continued)

Verification requires more than refreshing the page once. Test keyboard, touch, barcode scanner focus, and cancel paths because popup usability is more important in POS than in most back-office screens. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

POS Popups and Dialog Flow (continued)

In production, pos popups and dialog flow decisions affect performance, offline behavior, cache invalidation, and upgrade risk. In production, a popup should prevent mistakes without creating a line at the register. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W11 · KEY POINTS

What to remember

- Use popups for short high-value interruptions.
- Always handle cancel and confirm paths.
- Do not put long forms in POS popups.
- Keep focus behavior cashier-friendly.
- Validate data after the popup closes.

W11 · EXAMPLE 1

Confirmation before clearing order

Confirmation before clearing order

The destructive action is performed only from the confirmation callback.

```
async clearCurrentOrder() {
    const order = this.pos.get_order();
    if (!order || !order.get_orderlines().length) {
        return;
    }
    this.dialog.add(ConfirmationDialog, {
        title: "Clear order",
        body: "Remove all current order lines?",
        confirm: () => {
            for (const line of [...order.get_orderlines()]) {
                order.removeOrderline(line);
            }
        },
    });
# ... truncated for slide
```

Common mistakes

- Using a popup for information that could be a non-blocking notification.
- Forgetting the cancel path.
- Mutating orders before the user confirms.
- Creating popups that require too much typing at checkout.

> Caution — Validate fixes in a disposable database before production.

JavaScript Translations

- Mark JavaScript strings for translation.
- Avoid concatenation that breaks translators' context.
- Test translated POS and web screens.

> Lab target — Mark three JavaScript labels and one notification for translation, export terms, and verify them in another language.

JavaScript Translations

JavaScript Translations belongs to the client-side layer where user intent, browser state, and Odoo services meet. JavaScript translations matter because OWL and POS features often introduce user-facing labels outside Python and XML fields. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

JavaScript Translations (continued)

The design starts with ownership of state and boundaries. Write complete sentences for translators and pass variables as placeholders rather than building messages from fragments. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

JavaScript Translations (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Import _t or the appropriate translation helper, mark strings in components, and export translation terms through the normal Odoo i18n process. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

JavaScript Translations (continued)

Verification requires more than refreshing the page once. Test in a secondary language and look for layout overflow, untranslated strings, and strings generated dynamically from code values. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

JavaScript Translations (continued)

In production, javascript translations decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A translated customization feels like part of the product; untranslated JavaScript is one of the fastest ways to expose custom code quality. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W12 · KEY POINTS

What to remember

- Use _t for user-facing JavaScript strings.
- Translate complete messages, not fragments.
- Avoid concatenation with translated text.
- Test labels in narrow screens.
- Do not translate technical keys or registry names.

W12 · EXAMPLE 1

Translated notification

Translated notification

The full sentence is translatable and the course name is inserted as a placeholder.

```
/** @odoo-module **/

import { _t } from "@web/core/l10n/translation";

this.notification.add(
    _t("Course %(course)s was added to the training order.", {
        course: course.name,
    }),
    { type: "success" }
);
```

W12 · PITFALLS

Common mistakes

- Concatenating translated fragments.
- Forgetting strings inside POS-only components.
- Translating technical identifiers.
- Not testing how long translated labels affect layout.

> Caution — Validate fixes in a disposable database before production.

W13 · OBJECTIVES

POS RPC and Server Synchronization

- Call server methods from POS safely.
- Respect offline and cashier permissions.
- Batch data where possible.

> Lab target — Implement one POS RPC method for live data and document exactly what the cashier sees when the network is down.

POS RPC and Server Synchronization

POS RPC and Server Synchronization belongs to the client-side layer where user intent, browser state, and Odoo services meet. POS RPC is tempting for every feature, but online calls can slow checkout and fail during network interruptions. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

POS RPC and Server Synchronization (continued)

The design starts with ownership of state and boundaries. Decide whether data must be loaded at session start, cached in POS, sent with the order, or fetched on demand. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

POS RPC and Server Synchronization (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Use services to call model methods, batch requests, and show clear failure feedback when the POS must be online for the operation. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

POS RPC and Server Synchronization (continued)

Verification requires more than refreshing the page once. Test disconnected network behavior and low permissions because POS often runs with users that differ from back-office managers. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

POS RPC and Server Synchronization (continued)

In production, pos rpc and server synchronization decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production POS integrations should minimize synchronous calls during payment and order validation. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W13 · KEY POINTS

What to remember

- Prefer session-loaded data for frequently used reference data.
- Use RPC only when freshness is required.
- Batch calls to avoid checkout delays.
- Handle offline failures explicitly.
- Server methods must check permissions.

W13 · EXAMPLE 1

Fetch live loyalty balance

Fetch live loyalty balance

The method calls the server only when a partner exists and gives a cashier-friendly warning on failure.

```
async getLoyaltyBalance(partnerId) {
    if (!partnerId) {
        return 0;
    }
    try {
        return await this.orm.call(
            "training.loyalty",
            "pos_get_balance",
            [partnerId],
            { context: this.pos.user.context }
        );
    } catch {
        this.notification.add("Loyalty balance is unavailable while offline.", {
            type: "warning",
# ... truncated for slide
```

Common mistakes

- Making one RPC call per product scan.
- Blocking payment on optional online data.
- Ignoring the POS user's access rights.
- Not deciding what happens offline.

> Caution — Validate fixes in a disposable database before production.

W14 · OBJECTIVES

Rendering Dynamic POS Data

- Load additional fields into POS data.
- Render dynamic data on products or order lines.
- Keep display values synchronized with order state.

> Lab target — Load a training_level field into POS products and render it on the product card with an empty-state fallback.

Rendering Dynamic POS Data

Rendering Dynamic POS Data belongs to the client-side layer where user intent, browser state, and Odoo services meet. Dynamic POS rendering lets cashiers see availability, warnings, course metadata, or customer-specific information at the point of action. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

Rendering Dynamic POS Data (continued)

The design starts with ownership of state and boundaries. First decide whether the data is static for the session, changes during the session, or belongs on the order for accounting and audit. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Rendering Dynamic POS Data (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Load required fields through POS data loaders or model extensions, store display-ready values on POS models, and render them through OWL templates. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Rendering Dynamic POS Data (continued)

Verification requires more than refreshing the page once. Test reloads, order restore, and product cache updates because stale POS data can create incorrect cashier decisions. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

Rendering Dynamic POS Data (continued)

In production, rendering dynamic pos data decisions affect performance, offline behavior, cache invalidation, and upgrade risk. In production, dynamic POS UI should display enough information to guide action without becoming a back-office report. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W14 · KEY POINTS

What to remember

– Session-loaded data is fast but can become stale.
– Order-line data should be stored if needed later.
– Templates should render prepared values.
– Reload and restore behavior must be tested.
– Do not overload product tiles with too much data.

W14 · EXAMPLE 1

Product tile showing course level

Product tile showing course level

The template extension displays an extra product field only when the POS product data includes it.

```
<t t-name="training.ProductCard" t-inherit="point_of_sale.ProductCard" t-inherit-mode="extension">
    <xpath expr="//div[hasclass('product-name')]" position="after">
        <div t-if="props.product.training_level" class="training-level">
            <t t-esc="props.product.training_level"/>
        </div>
    </xpath>
</t>
```

Common mistakes

- Rendering fields that were never loaded into POS.
- Showing stale stock or capacity without a refresh policy.
- Putting audit-critical values only in the UI.
- Making product cards unreadable with too many badges.

> Caution — Validate fixes in a disposable database before production.

W15 · OBJECTIVES

Clearing Order Lines and Mutating POS Orders

- Mutate current orders through POS model APIs.
- Avoid unsafe direct array changes.
- Protect destructive actions with confirmation.

> Lab target — Add a clear-order control that confirms first, removes lines through the order API, and verifies totals return to zero.

Clearing Order Lines and Mutating POS Orders

Clearing Order Lines and Mutating POS Orders belongs to the client-side layer where user intent, browser state, and Odoo services meet. POS order mutation must use framework methods so totals, taxes, discounts, and UI state remain consistent. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

Clearing Order Lines and Mutating POS Orders (continued)

The design starts with ownership of state and boundaries. Before clearing or changing lines, decide whether the action is reversible and whether a manager confirmation is required. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Clearing Order Lines and Mutating POS Orders (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Iterate over a copy of the current order lines and remove them with the POS order API rather than splicing internal arrays. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Clearing Order Lines and Mutating POS Orders (continued)

Verification requires more than refreshing the page once. Test lines with discounts, lots, taxes, notes, and combo products because destructive actions often fail on less common line types. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

Clearing Order Lines and Mutating POS Orders (continued)

In production, clearing order lines and mutating pos orders decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A safe mutation preserves the invariants that payment, receipts, and order export depend on. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W15 · KEY POINTS

What to remember

– Use order APIs, not internal array mutation.
– Copy line arrays before removing while iterating.
– Confirm destructive actions.
– Recompute totals through framework behavior.
– Test special line types.

W15 · EXAMPLE 1

Clear all order lines safely

Clear all order lines safely

The method iterates over a copy and delegates removal to the POS order API.

```
clearOrderLines(order) {
    if (!order) {
        return;
    }
    for (const line of [...order.get_orderlines()]) {
        order.removeOrderline(line);
    }
    this.notification.add("The order was cleared.", { type: "success" });
}
```

Common mistakes

- Splicing order.orderlines directly.
- Removing lines while iterating over the live array.
- Skipping confirmation for destructive cashier actions.
- Forgetting special cases such as lot-tracked products.

> Caution — Validate fixes in a disposable database before production.

4 W16 – W20

OWL, POS & External APIs · teaching block 4

W16 · OBJECTIVES

XML-RPC Integration Fundamentals

– Authenticate with Odoo over XML-RPC.
– Call model methods through execute_kw.
– Understand database, uid, password, model, method, and args.

> Lab target — Create a script that authenticates over XML-RPC and reads the first ten confirmed courses with only three fields.

W16 · TEACHING 1

XML-RPC Integration Fundamentals

XML-RPC Integration Fundamentals belongs to the client-side layer where user intent, browser state, and Odoo services meet. XML-RPC remains a practical integration interface for scripts, legacy systems, and training labs. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

XML-RPC Integration Fundamentals (continued)

The design starts with ownership of state and boundaries. Design integrations around explicit service users with narrow groups, not administrator credentials copied into scripts. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

XML-RPC Integration Fundamentals (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Authenticate against common, then call object.execute_kw with model, method, positional args, and keyword options such as fields and limit. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

XML-RPC Integration Fundamentals (continued)

Verification requires more than refreshing the page once. Verify access rights and record rules with the same service user because XML-RPC observes normal Odoo security unless code uses sudo internally. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

XML-RPC Integration Fundamentals (continued)

In production, xml-rpc integration fundamentals decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A production XML-RPC integration should have credentials, logging, retries, and data validation treated like any other external system. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W16 · KEY POINTS

What to remember

- Authenticate on /xmlrpc/2/common.
- Call models through /xmlrpc/2/object execute_kw.
- Use service users with limited groups.
- XML-RPC respects Odoo access rules.
- Never hard-code production passwords in code.

W16 · EXAMPLE 1

Python XML-RPC login and search_read

Python XML-RPC login and search_read

The script authenticates once and calls search_read as the integration user.

```
import xmlrpc.client

url = "https://odoo.example.com"
db = "prod"
username = "integration@example.com"
password = "change-me"

common = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/common")
uid = common.authenticate(db, username, password, {})

models = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/object")
courses = models.execute_kw(
    db, uid, password,
    "training.course", "search_read",
# ... truncated for slide
```

Common mistakes

- Using the admin account for integrations.
- Forgetting that record rules affect API results.
- Fetching all fields and all records by default.
- Logging passwords or full credential URLs.

> Caution — Validate fixes in a disposable database before production.

W17 · OBJECTIVES

External API Authentication

- Select an authentication pattern for external integrations.
- Store secrets outside source code.
- Validate callbacks and inbound requests.

> Lab target — Design an authentication plan for one outbound API and one inbound webhook, including where each secret is stored.

External API Authentication

External API Authentication belongs to the client-side layer where user intent, browser state, and Odoo services meet. External APIs add another trust boundary around Odoo, so authentication design is part of the feature, not deployment cleanup. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

External API Authentication (continued)

The design starts with ownership of state and boundaries. Choose API keys, OAuth2, signed webhooks, or basic authentication based on the provider and risk of the data being exchanged. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

External API Authentication (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Store secrets in system parameters, environment variables, or a secret manager, and wrap outbound calls in a service class or model method. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

External API Authentication (continued)

Verification requires more than refreshing the page once. Test expired credentials, revoked tokens, replayed webhooks, and provider downtime so the integration fails safely. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

External API Authentication (continued)

In production, external api authentication decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production API authentication should be observable: operators need enough logs to diagnose failures without exposing secrets. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

What to remember

– Do not commit API credentials.
– Use least-privilege provider scopes.
– Validate webhook signatures when available.
– Handle token expiry deliberately.
– Log identifiers, not secrets.

W17 · EXAMPLE 1

Signed outbound request helper

Signed outbound request helper

The token is read from configuration at runtime and never appears in the source file.

```
import requests
from odoo import models


class TrainingExternalClient(models.AbstractModel):
    _name = "training.external.client"
    _description = "Training External API Client"

    def _headers(self):
        token = self.env["ir.config_parameter"].sudo().get_param("training.api_token")
        return {"Authorization": f"Bearer {token}", "Accept": "application/json"}

    def fetch_certificates(self):
        response = requests.get(
# ... truncated for slide
```

Common mistakes

- Committing secrets to the repository.
- Using broad OAuth scopes because they are easier during testing.
- Accepting inbound webhooks without signature validation.
- Logging Authorization headers during debugging.

> Caution — Validate fixes in a disposable database before production.

W18 · OBJECTIVES

Fetch and Export Workflows with XML-RPC

- Export selected records over XML-RPC.
- Paginate large result sets.
- Make fetch jobs restartable.

> Lab target — Write a paginated export for enrollments and simulate a restart after two batches.

W18 · TEACHING 1

Fetch and Export Workflows with XML-RPC

Fetch and Export Workflows with XML-RPC belongs to the client-side layer where user intent, browser state, and Odoo services meet. Export integrations fail in production when they assume the dataset is small and the network is perfect. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

Fetch and Export Workflows with XML-RPC (continued)

The design starts with ownership of state and boundaries. Define the export domain, fields, ordering, and checkpoint strategy before writing the loop. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

W18 · TEACHING 3

Fetch and Export Workflows with XML-RPC (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Use search with offset or keyset-style domains, read only required fields, and persist the last successful checkpoint outside the transient process. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Fetch and Export Workflows with XML-RPC (continued)

Verification requires more than refreshing the page once. Verify duplicates, skipped records, and interrupted runs by killing the process midway and restarting it against the same database. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

W18 · TEACHING 5

Fetch and Export Workflows with XML-RPC (continued)

In production, fetch and export workflows with xml-rpc decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A robust export job values correctness over raw speed because downstream systems often cannot detect missing records. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

What to remember

- Export only required fields.
- Paginate results instead of fetching everything.
- Use deterministic ordering.
- Persist checkpoints for restartability.
- Design for duplicates or idempotent downstream writes.

W18 · EXAMPLE 1

Paged XML-RPC export

Paged XML-RPC export

The loop exports in deterministic batches instead of loading the complete table at once.

```
offset = 0
batch_size = 100
while True:
    rows = models.execute_kw(
        db, uid, password,
        "training.enrollment", "search_read",
        [[["write_date", ">=", "2026-01-01 00:00:00"]]],
        {
            "fields": ["id", "student_id", "course_id", "state", "write_date"],
            "order": "id",
            "offset": offset,
            "limit": batch_size,
        },
    )
# ... truncated for slide
```

Common mistakes

- Exporting all records into memory.
- Using no order while paginating.
- Not handling retries or partial downstream failure.
- Exporting display names when stable ids are required.

> Caution — Validate fixes in a disposable database before production.

W19 · OBJECTIVES

CRUD Operations over XML-RPC

- Create, read, update, and delete records through XML-RPC.
- Pass relational field commands correctly.
- Respect constraints and security during API writes.

> Lab target — Create a course and enrollment over XML-RPC, update one field, and attempt an invalid write to confirm constraints are enforced.

CRUD Operations over XML-RPC

CRUD Operations over XML-RPC belongs to the client-side layer where user intent, browser state, and Odoo services meet. XML-RPC CRUD uses the same ORM methods that server code uses, so constraints, onchange gaps, access rights, and record rules all matter. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

CRUD Operations over XML-RPC (continued)

The design starts with ownership of state and boundaries. Design write payloads from the model contract, not from the current form layout, because UI-only onchange logic may not run during API calls. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

CRUD Operations over XML-RPC (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Use create, write, unlink, search_read, and relational command lists for one2many or many2many fields when needed. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

CRUD Operations over XML-RPC (continued)

Verification requires more than refreshing the page once. Verify bad payloads, duplicate submissions, and permission failures because integrations often retry after ambiguous network errors. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

CRUD Operations over XML-RPC (continued)

In production, crud operations over xml-rpc decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production CRUD integrations should be idempotent where possible and should log remote ids, local ids, and operation outcomes. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W19 · KEY POINTS

What to remember

- create returns the new record id.
- write returns True when the update succeeds.
- unlink deletes records if access and rules allow it.
- Relational commands use tuple-like command lists.
- Onchange behavior is not a substitute for server validation.

W19 · EXAMPLE 1

Create and update enrollment

Create and update enrollment

The API creates a draft enrollment and then updates its state through normal ORM security and constraints.

```
enrollment_id = models.execute_kw(
    db, uid, password,
    "training.enrollment", "create",
    [{
        "student_id": student_id,
        "course_id": course_id,
        "state": "draft",
    }],
)

models.execute_kw(
    db, uid, password,
    "training.enrollment", "write",
    [[enrollment_id], {"state": "confirmed"}],
# ... truncated for slide
```

Common mistakes

- Expecting form onchange methods to run during create.
- Sending display names instead of database ids for many2one fields.
- Deleting records with unlink when archiving would preserve audit history.
- Retrying creates without an external idempotency key.

> Caution — Validate fixes in a disposable database before production.

W20 · OBJECTIVES

Postman Authentication and CRUD Testing

– Use Postman to document and test API calls.

– Authenticate against Odoo endpoints deliberately.

– Build repeatable CRUD request collections.

> Lab target — Build a Postman collection with login, search_read, create, write, and negative-access requests using environment variables only.

Postman Authentication and CRUD Testing

Postman Authentication and CRUD Testing belongs to the client-side layer where user intent, browser state, and Odoo services meet. Postman is useful for training and integration discovery because it makes request shape, authentication, and responses visible. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

Postman Authentication and CRUD Testing (continued)

The design starts with ownership of state and boundaries. For Odoo JSON-RPC or controller endpoints, define environment variables for base URL, database, credentials, token, and record ids. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Postman Authentication and CRUD Testing (continued)

Implementation should follow Odoo's asset, registry, service, and template patterns. Create a collection that authenticates, stores returned values in variables, and then performs create, read, update, and delete or archive requests. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Postman Authentication and CRUD Testing (continued)

Verification requires more than refreshing the page once. Verify negative cases such as missing auth, wrong database, invalid ids, and insufficient rights because a collection that only proves success is incomplete. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

Postman Authentication and CRUD Testing (continued)

In production, postman authentication and crud testing decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A good Postman collection becomes living integration documentation when examples, tests, and environment variables are kept clean. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

W20 · KEY POINTS

What to remember

- Use environments for base URL and credentials.
- Store ids from responses for later requests.
- Add negative authorization tests.
- Do not export real secrets in shared collections.
- Document request purpose in the collection.

W20 · EXAMPLE 1

JSON-RPC body for search_read

JSON-RPC body for search_read

The request uses Postman environment variables so the same collection can run against staging or local databases.

```
{
  "jsonrpc": "2.0",
  "method": "call",
  "params": {
    "service": "object",
    "method": "execute_kw",
    "args": [
      "{{db}}",
      {{uid}},
      "{{password}}",
      "training.course",
      "search_read",
      [[["state", "=", "confirmed"]]],
      {"fields": ["name", "start_date"], "limit": 5}
# ... truncated for slide
```

Common mistakes

- Exporting collections with real passwords.
- Hard-coding record ids that differ by database.
- Testing only successful CRUD operations.
- Confusing web session authentication with integration authentication.

> Caution — Validate fixes in a disposable database before production.

Postman collection sections

Folder	Purpose
Auth	Login and store uid or token
Read	search_read and field checks
Write	create and update examples
Negative	missing auth and forbidden role tests

WRAP-UP

You should now be able to...

- W01 OWL Architecture and Reactive Rendering
- W02 Component Structure and Lifecycle Hooks
- W03 useState, useRef, Events, and Props
- W04 QWeb XML Directives: t-if, t-foreach, and t-model
- W05 Custom Form Field Widgets
- W06 Overriding JavaScript Components with the Patch Utility
- ... and 14 more lessons in this deck

> Remember — Complete outstanding labs before starting the next section.

OWL POS RPC XML-RPC

NEXT STEP

Complete the section labs

Open the next presentation deck

Section 06 · OWL / POS / API